

6. Code Generation

6.1 Overview

6.2 The MicroJava VM

6.3 Code Buffer

6.4 Operands

6.5 Expressions

6.6 Assignments

6.7 Jumps and Labels

6.8 Control Structures

6.9 Methods



Tasks of code generation

Generation of machine instructions

- selecting the appropriate instructions
- selecting the appropriate addressing modes

Translation of control structures (if, while, ...) into jumps

Management of stack frames for local variables

Maybe some optimizations

Output of the object file

Common strategy of code generation



1. Study the target machine

registers, data formats, addressing modes, instructions, instruction formats, ...

2. Design the run-time data structures

layout of stack frames, layout of the global data area, layout of heap objects, ...

3. Implement the code buffer

instruction encoding, instruction patching, ...

4. Implement register allocation

irrelevant for MicroJava, because we have a stack machine

5. Implement code generation (in the following order)

- load values (to the expression stack)
- process designators (x.y, a[i], ...)
- translate expressions
- manage labels and jumps
- translate statements
- translate methods and parameter passing

6. Code Generation

6.1 Overview

6.2 The MicroJava VM

6.3 Code Buffer

6.4 Operands

6.5 Expressions

6.6 Assignments

6.7 Jumps and Labels

6.8 Control Structures

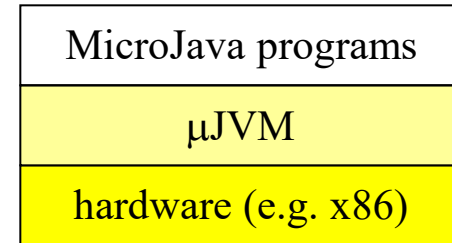
6.9 Methods

Architecture of the MicroJava VM (μ JVM)



What is a virtual machine (VM)?

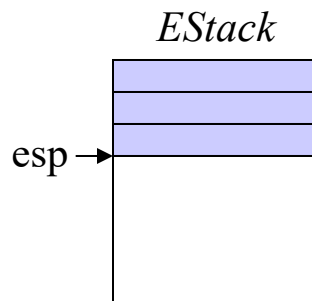
- A software CPU
- instructions are interpreted (or "jitted")
- e.g: Java VM, Smalltalk VM, Pascal P-Code



Goal: portability of programs

The μ JVM is a stack machine

- no registers
- instead it has an *expression stack* (onto which values are loaded)



word array (1 word = 4 bytes)
need not be big (e.g. 32 words $\equiv$ 32 registers)

esp ... expression stack pointer

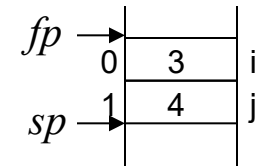
How a stack machine works



Example

statement $i = i + j * 5;$

assume the following values of i and j



Simulation

instructions *EStack*

load0

3

load variable from address 0 (i.e. i)

load1

3	4
---	---

load variable from address 1 (i.e. j)

const5

3	4	5
---	---	---

load constant 5

mul

3	20
---	----

multiply the two topmost stack elements

add

23

add the two topmost stack elements

store0

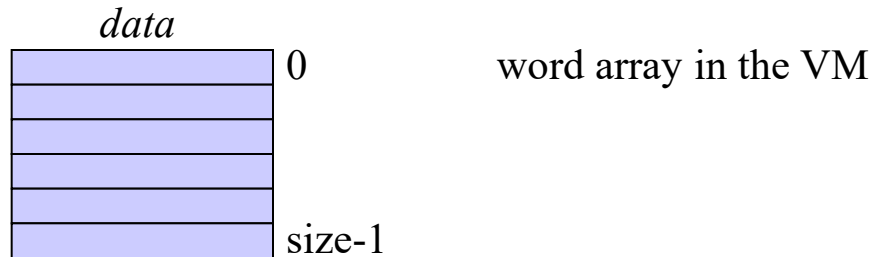
store the topmost stack element to address 0

At the end of every statement the expression stack is empty again!



Data areas of the μJVM

Global variables



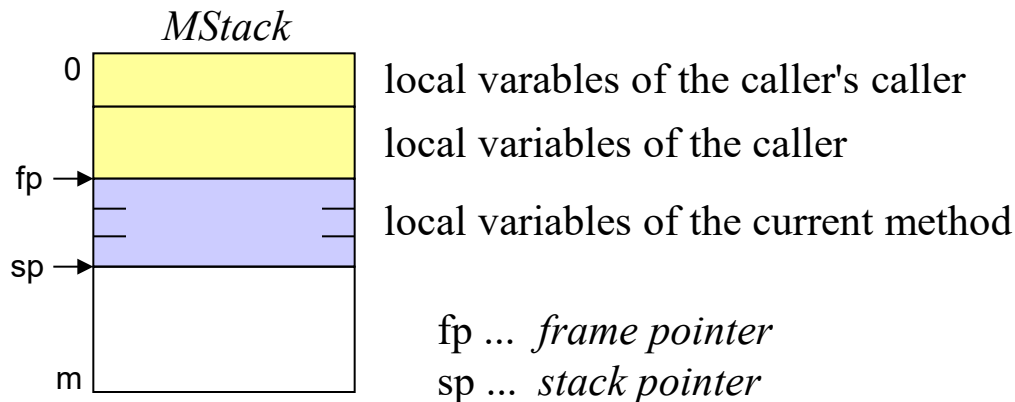
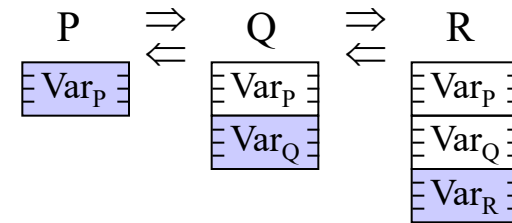
- area of fixed size
- global variables live during the whole program execution
- each variable occupies 1 word (4 bytes)
- global variables are addressed by word numbers
e.g., *getstatic 2* loads the variable at address 2 from *data* to *EStack*

Data areas of the μ JVM



Local variables

- are allocated in a *stack frame*
- every method invocation has its own stack frame
- frames are managed in a stack-like way



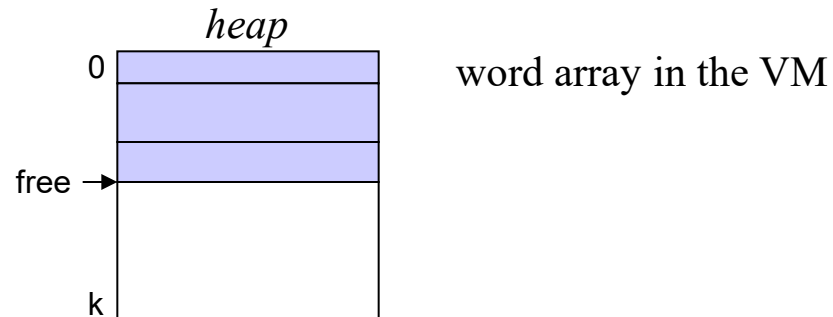
- local variables are addressed relative to *fp*
- each variable occupies 1 word (4 bytes)
- local variables are addressed by word numbers
e.g., *load0* loads the variable at offset 0 from *fp* to *EStack*

Data areas of the μ JVM



Heap

- contains class and array objects



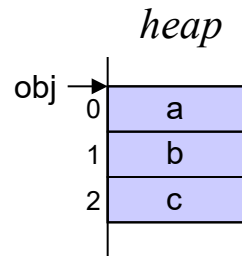
- New objects are allocated at the position *free* (and *free* is incremented); this is done by the VM instructions *new* and *newarray*
- Objects are never deallocated in MicroJava (no garbage collector)
- Pointers are word addresses relative to the beginning of the heap

Data areas of the μ JVM



class objects

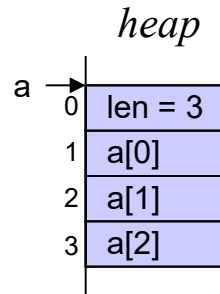
```
class T {  
    int a, b;  
    char c;  
}  
T obj = new T;
```



- each field occupies 1 word (4 bytes)
- addressed by word numbers relative to *obj*

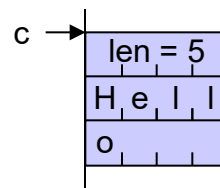
array objects

```
int[] a;  
a = new int[3];
```



- array length is stored in the array object
- each element occupies 1 word (4 bytes)

```
char[] c = new char[5];
```



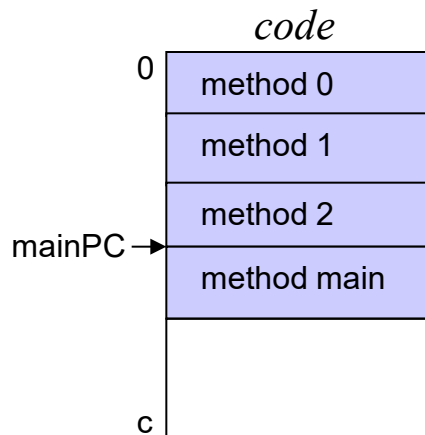
- *char* arrays are byte arrays
- but their size is a multiple of 4 bytes

Code area of the μ JVM



Code

- byte array of fixed size
- methods are allocated consecutively
- *mainPC* points to the *main()* method



byte array in the VM

special registers of the VM

fp frame pointer
sp stack pointer (*MStack*)
esp stack pointer (*EStack*)
pc program counter

Instruction set of the μ JVM



Bytecodes (similar to Java bytecodes)

- very compact: most instructions are just 1 byte long
- untyped (the Java VM encodes operand types in instructions)

MicroJava

load0
load1
add

Java

iload0
iload1
iadd

fload0
fload1
fadd

reason: the Java bytecode verifier can use the operand types to check the integrity of the program

Instruction format

very simple compared to Intel, PowerPC or SPARC

Code = {Instruction}.

Instruction = opcode {operand}.

opcode ... 1 byte

operand ... 1, 2 or 4 bytes

Examples

0 operands	add	has 2 implicit operands on the expression stack
1 operand	load 7	
2 operands	enter 0, 2	method entry

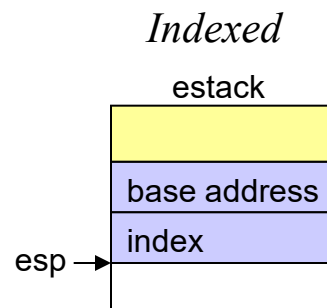
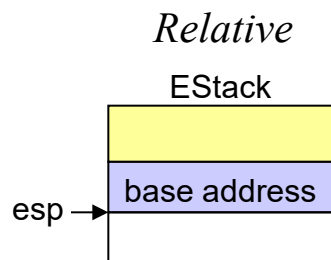
Instruction set of the μJVM



Addressing modes

Ways how operands can be accessed

<i>addressing mode</i>	<i>example</i>	
• Immediate	const 7	for constants
• Local	load 3	for local variables on <i>MStack</i>
• Static	getstatic 3	for global variables in <i>data</i>
• Stack	add	for loaded values on <i>EStack</i>
• Relative	getfield 3	for object fields (load $heap[pop() + 3]$)
• Indexed	aload	for array elements (load $heap[pop() + pop() + 1]$)



Instruction set of the μJVM



Load/store of local variables

load b	, val	<u>Load</u> push(local[b]);
loadn	, val	<u>Load</u> (n = 0..3) push(local[n]);
store b	..., val ...	<u>Store</u> local[b] = pop();
storen	..., val ...	<u>Store</u> (n = 0..3) local[n] = pop();

operand sizes

b ... byte

s ... short (2 bytes)

w ... word (4 bytes)

Load/store of global variables

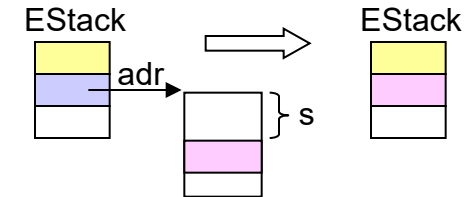
getstatic s	, val	<u>Load static variable</u> push(data[s]);
putstatic s	..., val ...	<u>Store static variable</u> data[s] = pop();

Instruction set of the μJVM



Load/store of object fields

getfield	s	..., adr ..., val	<u>Load object field</u> adr = pop(); push(heap[adr+s]);
putfield	s	..., adr, val ...	<u>Store object field</u> val = pop(); adr = pop(); heap[adr+s] = val;

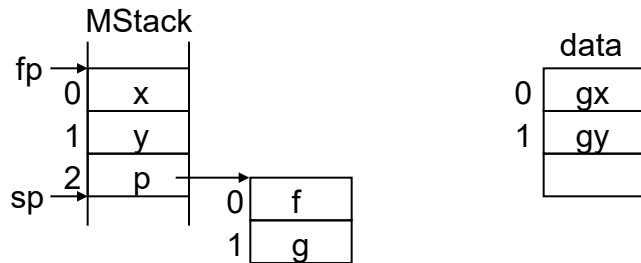


Loading constants

const	w	, val	<u>Load constant</u> push(w);
constn		, val	<u>Load constant</u> (n = 0..5) push(n);
const_m1		, val	<u>Load minus one</u> push(-1);



Examples: loading and storing



	<i>code</i>	<i>bytes</i>	<i>EStack</i>
x = y;	load1 store0	1 1	y -
gx = gy;	getstatic 1 putstatic 0	3 3	gy -
p.f = p.g;	load2 load2 getfield 1 putfield 0	1 1 3 3	p p p p p.g -



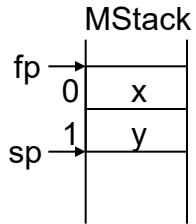
Instruction set of the μ JVM

Arithmetic

add	..., val1, val2 ..., val1+val2	<u>Add</u> push(pop() + pop());
sub	..., val1, val2 ..., val1-val2	<u>Subtract</u> push(-pop() + pop());
mul	..., val1, val2 ..., val1*val2	<u>Multiply</u> push(pop() * pop());
div	..., val1, val2 ..., val1/val2	<u>Divide</u> y = pop(); push(pop() / y);
rem	..., val1, val2 ..., val1%val2	<u>Remainder</u> y = pop(); push(pop() % y);
neg	..., val ..., -val	<u>Negate</u> push(-pop());
shl	..., val, x ..., val1	<u>Shift left</u> x = pop(); push(pop() << x);
shr	..., val, x ..., val1	<u>Shift right</u> x = pop(); push(pop() >> x);
inc b1, b2		<u>Increment local variable</u> local[b1] = local[b1] + b2;



Examples: arithmetic operations



	<i>code</i>	<i>bytes</i>	<i>EStack</i>
x + y * 3	load0	1	x
	load1	1	x y
	const3	1	x y 3
	mul	1	x y*3
	add	1	x+y*3
x++;	inc 0,1	3	-
x--;	inc 0,-1	3	-

Instruction set of the μJVM

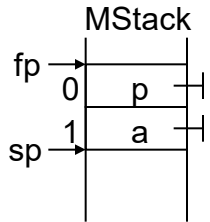


Object creation

new s	, adr	<u>New object</u> allocate area of s words; initialize area to all 0; push(adr(area));
newarray b	..., len ..., adr	<u>New array</u> len = pop(); if (b == 0) allocate byte array with <i>len</i> elements (+ length word); else if (b == 1) allocate word array with <i>len</i> elements (+ length word); initialize array to all 0; store <i>len</i> as the first word of the array; push(adr(array));



Examples: object creation



Assume: $\text{size}(\text{Person}) = 4 \text{ words}$

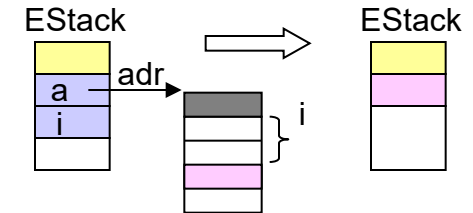
	<i>code</i>	<i>bytes</i>	<i>EStack</i>	
Person p = new Person;	new 4 store0	3 1	p -	
int[] a = new int[5];	const5 newarray 1 store1	1 2 1	5 a -	
char[] b = new char[6];	const 6 newarray 0 store2	5 2 1	6 b -	

Instruction set of the μJVM



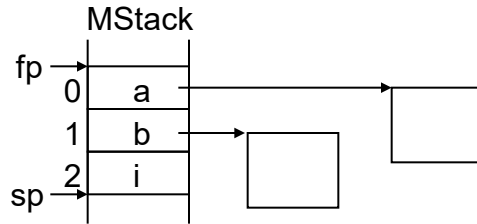
Array access

aload	..., adr, i ..., val	<u>Load array element</u> i = pop(); adr = pop(); push(heap[adr+1+i]);
astore	...,adr, i, val ...	<u>Store array element</u> val = pop(); i = pop(); adr = pop(); heap[adr+1+i] = val;
baload	..., adr, i ..., val	<u>Load byte array element</u> i = pop(); adr = pop(); x = heap[adr+1+i/4]; push(byte i%4 of x);
bastore	...,adr, i, val ...	<u>Store byte array element</u> val = pop(); i = pop(); adr = pop(); x = heap[adr+1+i/4]; set byte i%4 in x to val; heap[adr+1+i/4] = x;
arraylength	..., adr ..., len	<u>Get array length</u> adr = pop(); push(heap[adr]);





Example: array access



	<i>code</i>	<i>bytes</i>	<i>EStack</i>
a[i] = b[i+1];	load0	1	a
	load2	1	a i
	load1	1	a i b
	load2	1	a i b i
	const1	1	a i b i 1
	add	1	a i b i+1
	aload	1	a i b[i+1]
	astore	1	-



Instruction set of the μ JVM

Stack manipulation

pop	..., val ...	<u>Remove topmost stack element</u> dummy = pop();
dup	..., val ..., val, val	<u>Duplicate topmost stack element</u> x = pop(); push(x); push(x);
dup2	..., x, y ..., x, y, x, y	<u>Duplicate top two stack elements</u> y = pop(); x = pop(); push(x); push(y); push(x); push(y);

Jumps

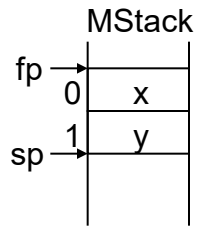
jmp s		<u>Jump unconditionally</u> pc = pc + s;
j<cond> s	..., x, y ...	<u>Jump conditionally</u> (eq,ne,lt,le,gt,ge) y = pop(); x = pop(); if (x cond y) pc = pc + s;

jump distance relative
to start of instruction

jeq
jne
jlt
jle
jgt
jge



Example: jumps



	<i>code</i>	<i>bytes</i>	<i>EStack</i>
if (x > y) ...	load0	1	x
	load1	1	x y
	jle ...	3	-

Instruction set of the μJVM



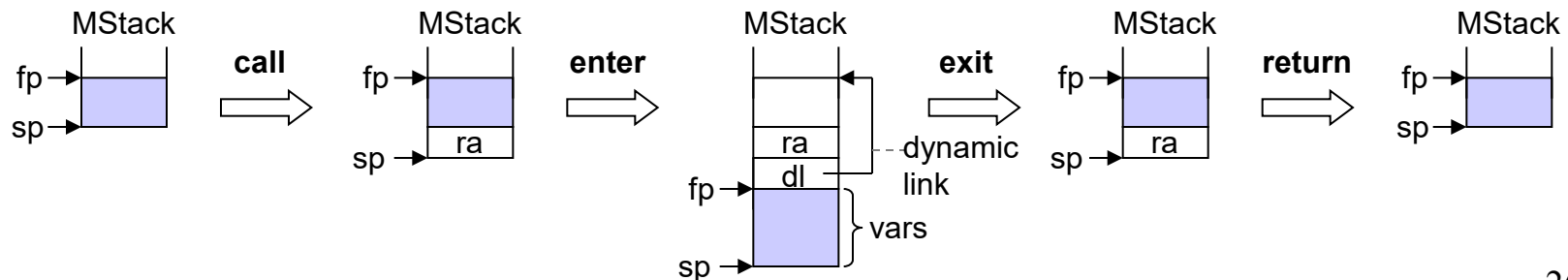
Method call

call	s	...	...	<u>Call method</u> PUSH(pc+3); pc = pc + s;
enter	b1, b2	...	...	<u>Enter method</u> pars = b1; vars = b2; // in words PUSH(fp); fp = sp; sp = sp + vars; initialize frame to 0; for (i=pars-1; i>=0; i--) local[i] = pop();
exit		...	...	<u>Exit method</u> sp = fp; fp = POP();
return		...	...	<u>Return</u> pc = POP();

PUSH and POP work on *MStack*

pars ...
nr. of parameters

vars ...
nr. of local variables (incl. parameters)





Instruction set of the μ JVM

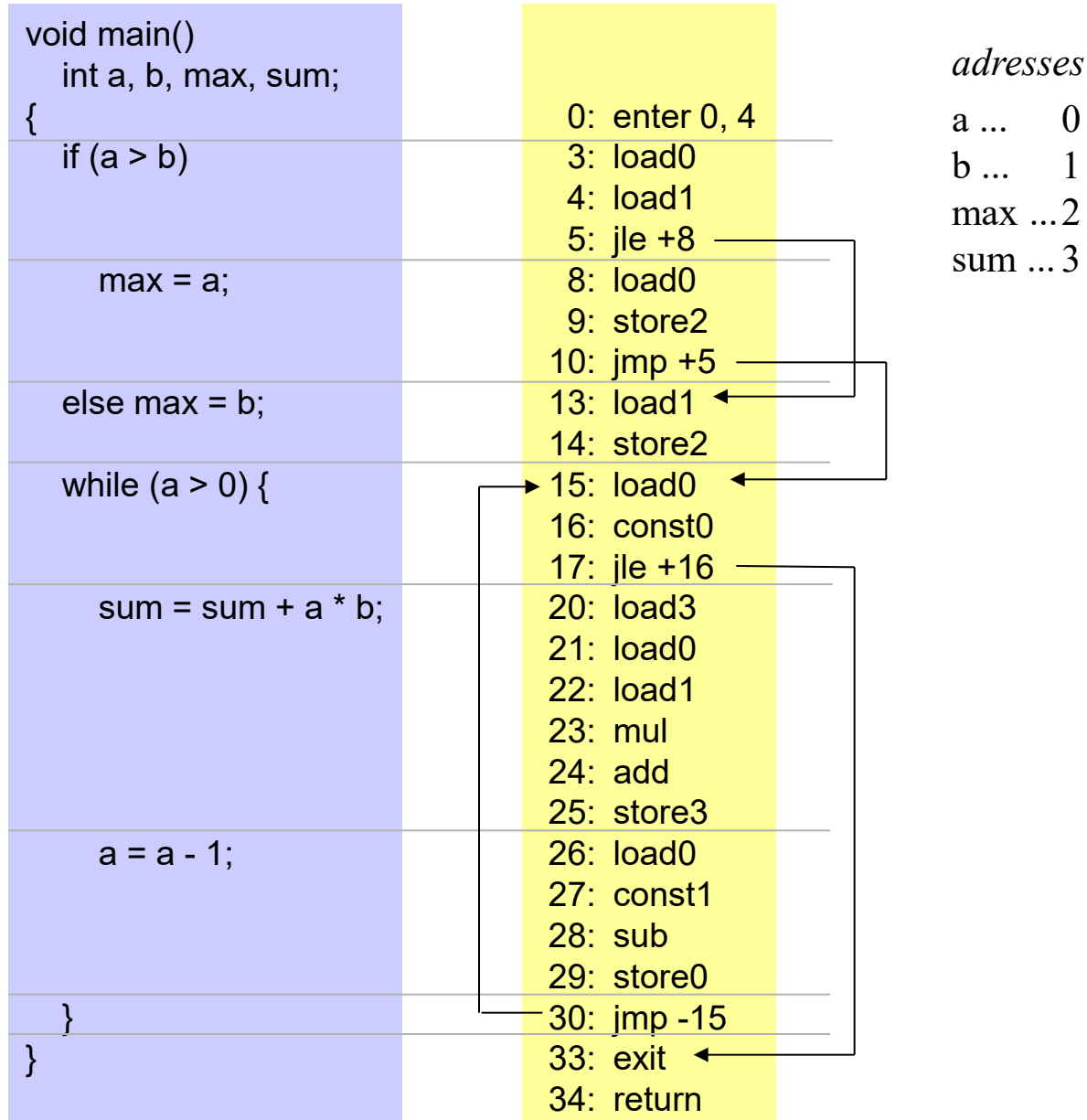
Input/output

read	, val	<u>Read integer</u> x = readInt(); push(x);	input from <i>System.in</i> output to <i>System.out</i>
print	..., val, width ...	<u>Print integer</u> w = pop(); writeInt(pop(), w);	Prints <i>val</i> right-aligned in a field of <i>w</i> characters
bread	, val	<u>Read character</u> ch = readChar(); push(ch);	
bprint	..., val, width ...	<u>Print character</u> w = pop(); writeChar(pop(), w);	

Run-time errors

trap	b		<u>Throw exception</u> print error message based on b; stop execution;
-------------	---	------------	--

Example



6. Code Generation

6.1 Overview

6.2 The MicroJava VM

6.3 Code Buffer

6.4 Operands

6.5 Expressions

6.6 Assignments

6.7 Jumps and Labels

6.8 Control Structures

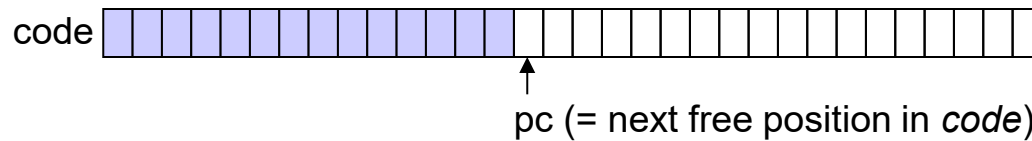
6.9 Methods

Code buffer



Data structure

byte array in memory, because some instructions have to be patched later



Emitting instructions

simple, because MicroJava has a simple instruction format

```
class Code {  
    private static byte[] code = new byte[3000];  
    public static int pc = 0;  
    public static void put (int x); {  
        code[pc++] = (byte)x;  
    }  
    public static void put2 (int x) {  
        put(x >> 8); put(x);  
    }  
    public static void put4 (int x) {  
        put2(x >> 16); put2(x);  
    }  
    ...  
}
```

instruction codes are declared in class *Code*

```
static final int  
    load      = 1,  
    load0     = 2,  
    load1     = 3,  
    load2     = 4,  
    load3     = 5,  
    store     = 6,  
    store0    = 7,  
    store1    = 8,  
    store2    = 9,  
    store3    = 10,  
    getstatic = 11,  
    ... ;
```

e.g., emitting *load 7*

```
Code.put(Code.load);  
Code.put(7);
```

e.g.: emitting *load2*

```
Code.put(Code.load0 + 2);
```

6. Code Generation

6.1 Overview

6.2 The MicroJava VM

6.3 Code Buffer

6.4 Operands

6.5 Expressions

6.6 Assignments

6.7 Jumps and Labels

6.8 Control Structures

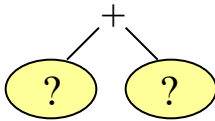
6.9 Methods

Operands during code generation



Example

we want to add two values



desired code pattern

load operand 1
load operand 2 } how should that be done?
add

Depending on the operand kind we must generate different load instructions

<i>operand kind</i>	<i>instruction to be generated</i>
• constant	const val
• local variable	load adr
• global variable	getstatic adr
• object field	getfield adr
• array element	aload
• loaded value on the stack	---

We need a descriptor, which gives us all the necessary information about operands

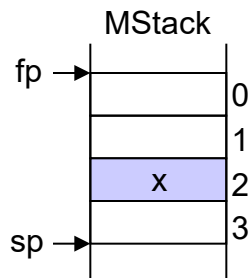


Operand descriptors

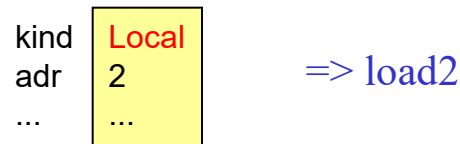
Hold information about variables, constants and expressions

Example

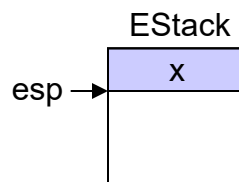
Local variable *x* in a stack frame



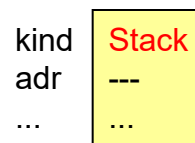
described by the following operand descriptor



After loading the value with *load2* it is on *EStack*



described by the following operand descriptor

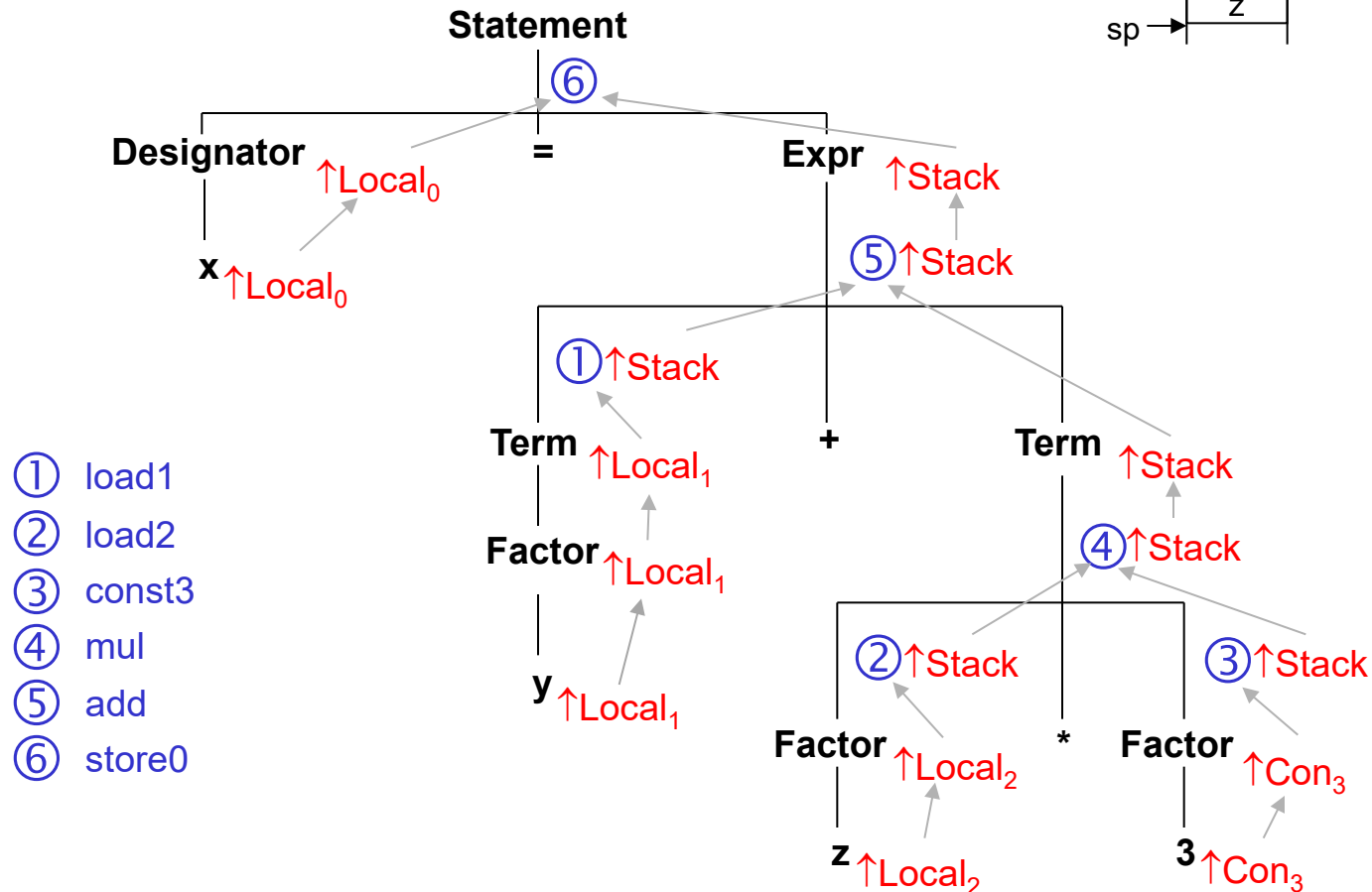
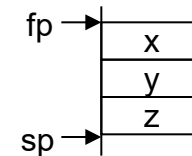


Example: processing of operands



Most parsing methods return operands (as a result of their translation process)

Example: translating the assignment `x = y + z * 3;`



Operand kinds



<i>operand kind</i>	<i>operand code</i>	<i>info about operands</i>	
constant	Con = 0	constant value	
local variable	Local = 1	address	
global variable	Static = 2	address	
value on EStack	Stack = 3	---	
object field	Fld = 4	offset	
array element	Elem = 5	---	
method	Meth = 6	address, method obj.	

How to find the necessary operand kinds



addressing modes

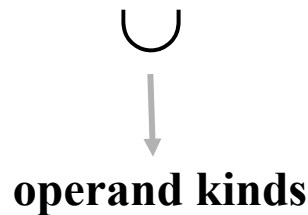
depending on the target machine

- Immediate
- Local
- Static
- Stack
- Relative
- Indexed

object kinds

depending on the source language

- Con
- Var
- Type
- Meth



- Con
- Local
- Static
- Stack
- Fld
- Elem
- Meth

We do not need *Type* operands in MicroJava,
because types do not occur as operands
(no type casts)

Class Operand



```
class Operand {
    static final int Con = 0, Local = 1, Static = 2, Stack = 3, Fld = 4, Elem = 5, Meth = 6;

    int    kind;    // Con, Local, Static, Stack, Fld, Elem, Meth
    Struct type;    // type of the operand
    int    val;     // Con: constant value
    int    adr;     // Local, Static, Fld, Meth: address
    Obj    obj;     // Meth: method object
}
```

Constructors for creating operands

```
public Operand (Obj obj) {
    type = obj.type; val = obj.val; adr = obj.adr;
    switch (obj.kind) {
        case Obj.Con:    kind = Con; break;
        case Obj.Var:    if (obj.level == 0) kind = Static; else kind = Local;
                        break;
        case Obj.Meth:   kind = Meth; this.obj = obj; break;
        default:         error(" a type is not a valid operand ");
    }
}
```

creates an operand from
a symbol table object

```
public Operand (int val) {
    kind = Con; type = Tab.intType; this.val = val;
}
```

creates an operand from
a constant value



Loading values

given: a value described by an operand descriptor (Con, Local, Static, ...)

wanted: code that loads the value onto the expression stack

```
public static void load (Operand x) { // method of class Code
    switch (x.kind) {
        case Operand.Con:
            if (0 <= x.val && x.val <= 5) put(const0 + x.val);
            else if (x.val == -1) put(const_m1);
            else { put(const_); put4(x.val); }
            break;
        case Operand.Static:
            put(getstatic); put2(x.adr); break;
        case Operand.Local:
            if (0 <= x.adr && x.adr <= 3) put(load0 + x.adr);
            else { put(load); put(x.adr); }
            break;
        case Operand.Fld: // assert: object base address is on EStack
            put(getfield); put2(x.adr); break;
        case Operand.Elem: // assert: base address & index are on EStack
            if (x.type == Tab.charType) put(baload); else put(aload);
            break;
        case Operand.Stack: break; // nothing (already loaded)
        case Operand.Meth: error("cannot load a method");
    }
    x.kind = Operand.Stack;
}
```

Case analysis

depending on the operand kind we have to generate different load instructions

resulting operand is always a *Stack* operand

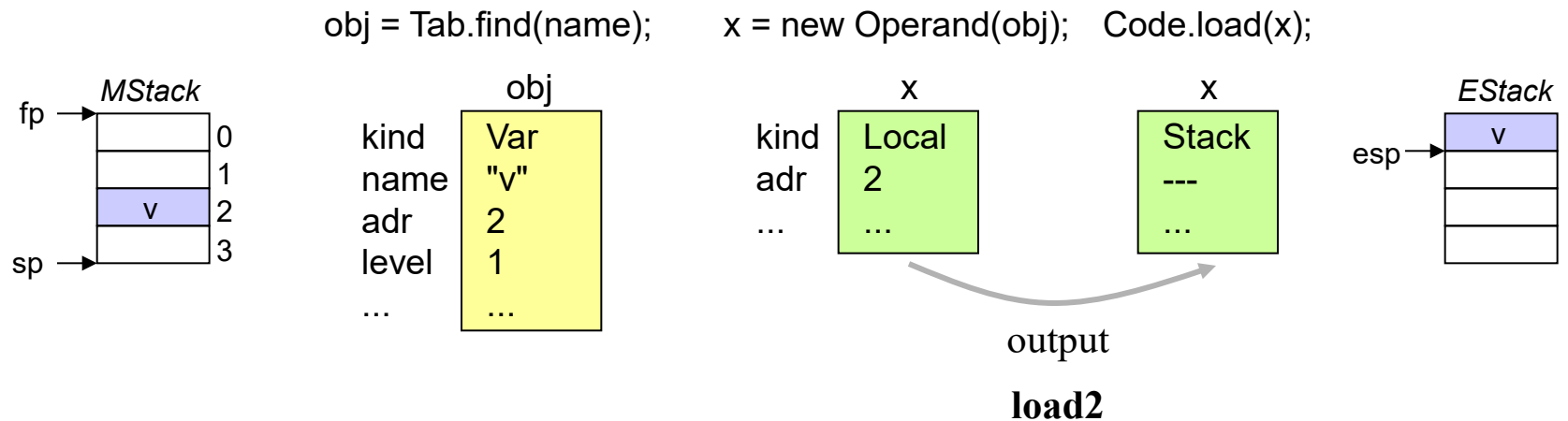


Example: loading variables

Description by an ATG

```
Factor <↑x>      (. String name; .)
= ident <↑name>  (. Obj obj = Tab.find(name);      // obj.kind = Var | Con
                  Operand x = new Operand(obj);    // x.kind = Local | Static | Con
                  Code.load(x);                    // x.kind = Stack
                  .) .
```

Visualization



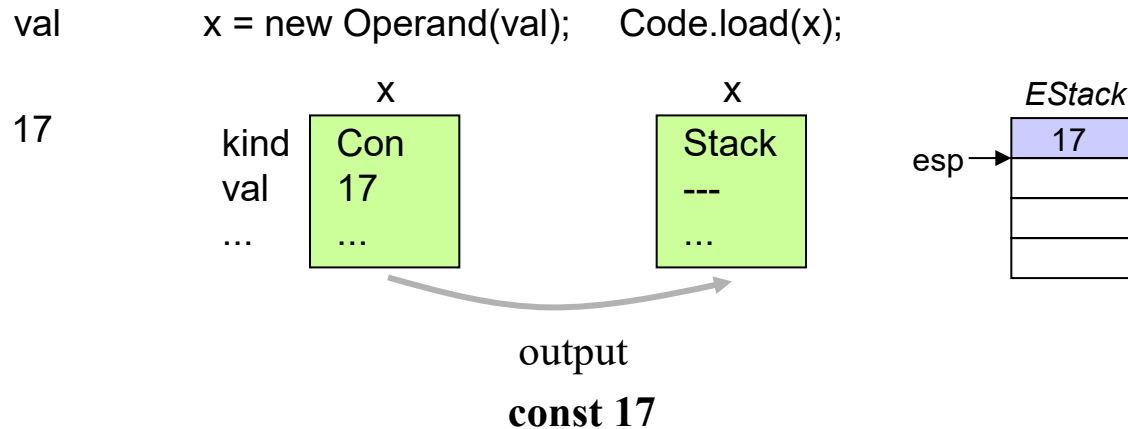


Example: loading constants

Description by an ATG

```
Factor <↑x>      (. int val; .)
= ...
| number <↑val>  (. Operand x = new Operand(val); // x.kind = Con
                  Code.load(x);                  // x.kind = Stack
                  .)
```

Visualization





Loading object fields

var.f

Context conditions (make sure that your compiler checks them)

$\text{Designator}_0 = \text{Designator}_1 \text{ "." ident .}$

- The type of Designator_1 must be a class.
- ident must be a field of Designator_1 .

Description by an ATG

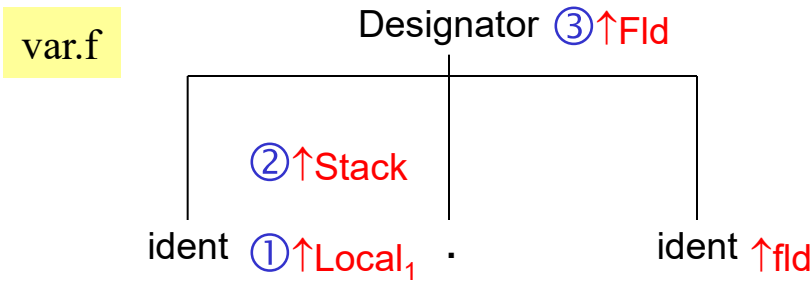
```
Designator <↑x>      (. String name, fName; .)
= ident <↑name>      (. Obj obj = Tab.find(name);
                      Operand x = new Operand(obj); .)
{ "." ident <↑fName>  (. if (x.type.kind == Struct.Class) {
                      Code.load(x);
                      Obj fld = Tab.findField(fName, x.type);
                      x.kind = Operand.Fld;
                      x.adr = fld.adr;
                      x.type = fld.type;
                      } else error(name + " is not an object"); .)
| ...
}.
```

looks up $fName$ in the
field list of $x.type$

creates a *Fld* operand

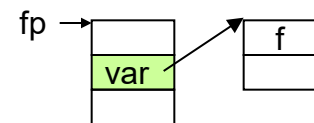
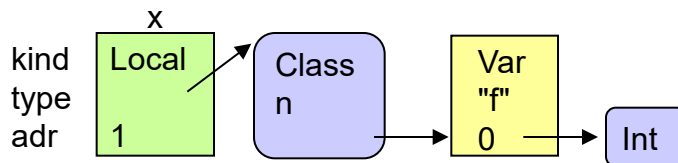


Example: loading object fields

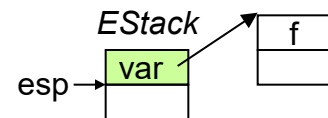
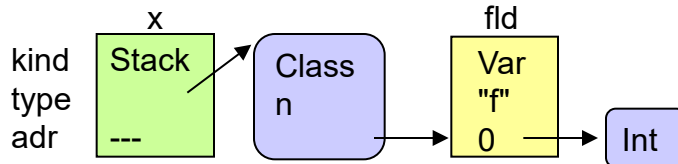


```
Designator <↑x>
= ident <↑name>
{ "." ident <↑fname>
  Code.load(x);
  Obj fld = Tab.findField(fname, x.type);
  x.kind = Operand.Fld;
  x.adr = fld.adr;
  x.type = fld.type;
} else error(name + " is not an object"); .
}.
```

① Operand x = new Operand(varObj);

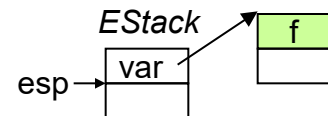
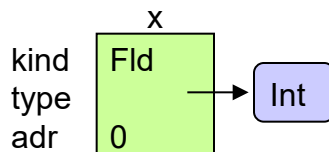


② Code.load(x);

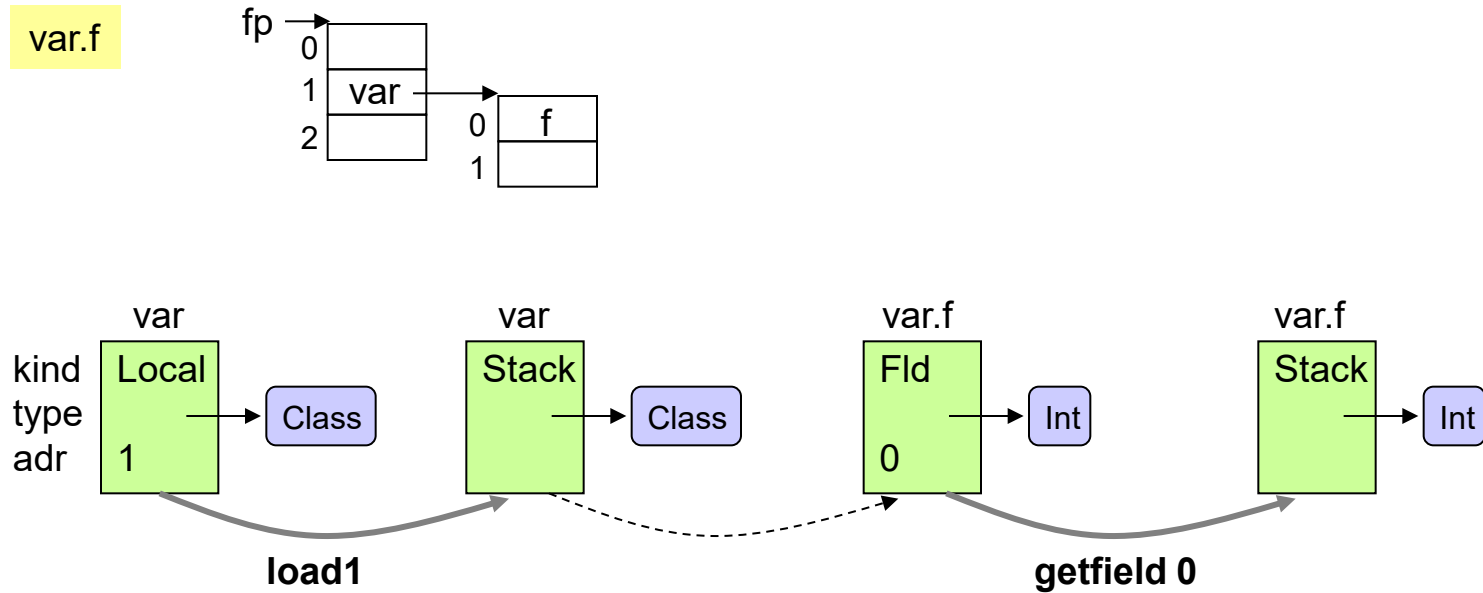


output
load1

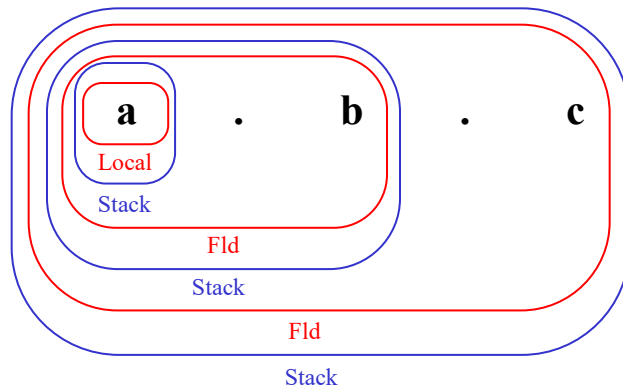
③ create a Fld operand from x and fld



Sequence of operands (1)



Sequence of operands (2)



load a
getfield b
getfield c

Loading array elements



a[i]

Context conditions

$\text{Designator}_0 = \text{Designator}_1 \text{ "[" Expr "]" } .$

- The type of Designator_1 must be an array.
- The type of Expr must be *int*.

Description by an ATG

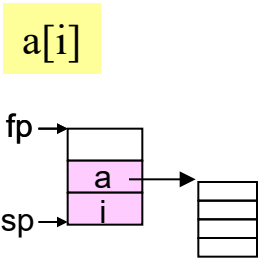
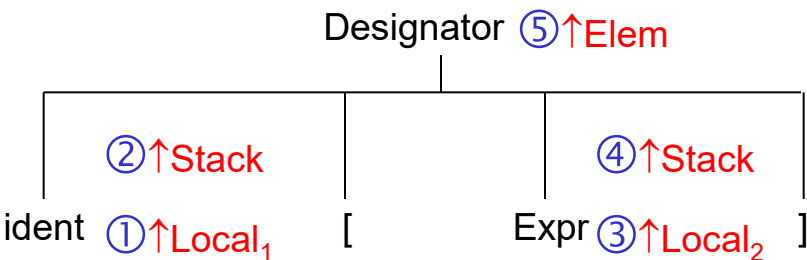
```
Designator <↑x>      (. String name; Operand x, y; .)
= ident <↑name>      (. Obj obj = Tab.find(name); x = new Operand(obj); .)
{ ...
| "["               (. Code.load(x); .)
  Expr <↑y>         (. if (x.type.kind == Struct.Arr) {
                    if (y.type != Tab.intType) error("index must be of type int");
                    Code.load(y);
                    x.kind = Operand.Elem; ← creates an Elem operand
                    x.type = x.type.elemType;
                    } else error(name + " is not an array"); .)

  "]"
}.

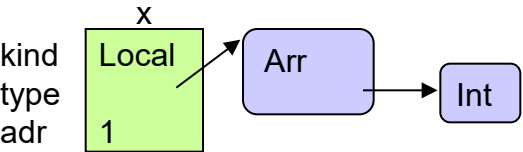
```



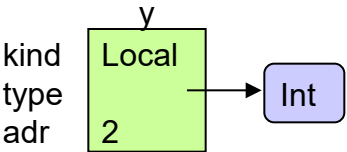
Example: loading array elements



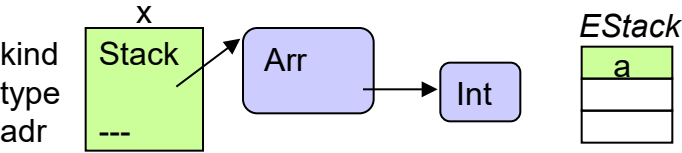
① Operand x = new Operand(obj);



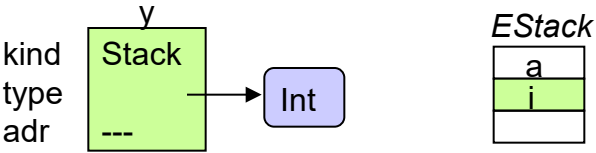
③ y = Expr();



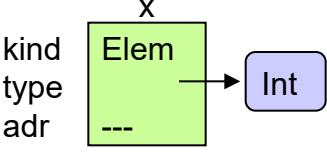
② Code.load(x);



④ Code.load(y);



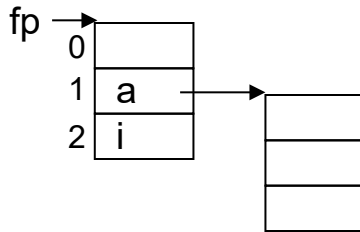
⑤ create an *Elem* operand from x and y



Sequence of operands (1)



a[i]



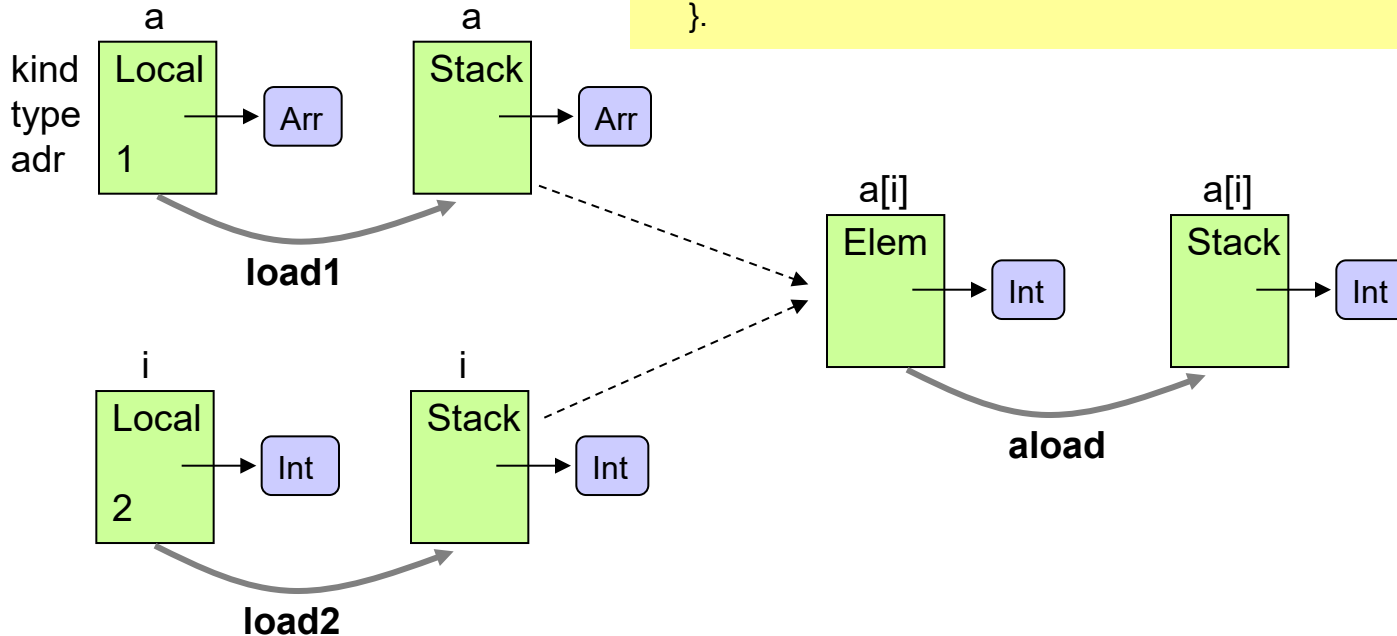
Designator $\langle \uparrow x \rangle$
 = ident $\langle \uparrow \text{name} \rangle$

```
{ ...
  "["
  Expr  $\langle \uparrow y \rangle$ 
```

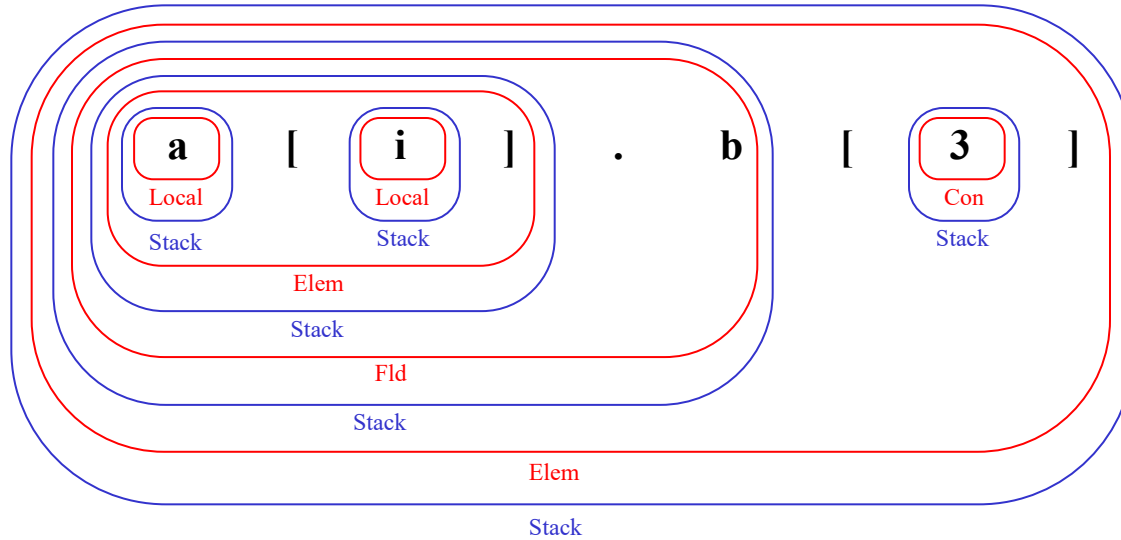
```
  "]"
}
```

```
(. String name; Operand x, y; .)
(. Obj obj = Tab.find(name); x = new Operand(obj); .)

{ ...
  "["
  (. Code.load(x); .)
  (. if (x.type.kind == Struct.Arr) {
      if (y.type.kind != Struct.Int)
        error("index must be of type int");
      Code.load(y);
      x.kind = Operand.Elem;
      x.type = x.type.elemType;
    } else error(name + " is not an array"); .)
```



Sequence of operands (2)



load a
load i
aload
getfield b
const3
aload

6. Code Generation

6.1 Overview

6.2 The MicroJava VM

6.3 Code Buffer

6.4 Operands

6.5 Expressions

6.6 Assignments

6.7 Jumps and Labels

6.8 Control Structures

6.9 Methods

Compiling expressions



Scheme for $x + y + z$

```
load x
load y
add
load z
add
```

Context conditions

Expr = "-" Term.

- *Term* must be of type *int*.

Expr₀ = Expr₁ Addop Term.

- Expr₁ and *Term* must be of type *int*.

Description by an ATG

```
Expr <↑x>      ( . Operand x, y; int op; . )
= ( Term <↑x>
  | "-" Term <↑x>
  )
  { ( "+"      ( . op = Code.add; . )
    | "-"      ( . op = Code.sub; . )
    )          ( . Code.load(x); . )
    Term <↑y>  ( . Code.load(y);
               if (x.type != Tab.intType || y.type != Tab.intType)
                 error("operands must be of type int");
               Code.put(op); . )
  }.
```

Compiling terms



$\text{Term}_0 = \text{Term}_1 \text{ Mulop Factor}.$

- Term_1 and Factor must be of type *int*.

```
Term <↑x>      (. Operand x, y; int op; .)
= Factor <↑x>
  { ( "*"        (. op = Code.mul; .)
    | "/"        (. op = Code.div; .)
    | "%"        (. op = Code.rem; .)
    )            (. Code.load(x); .)
    Factor <↑y>  (. Code.load(y);
                  if (x.type != Tab.intType || y.type != Tab.intType)
                      error("operands must be of type int");
                  Code.put(op); .)
  }.
```

Compiling factors



Factor = "new" ident.

- *ident* must denote a class.

Factor = "new" ident "[" Expr "].

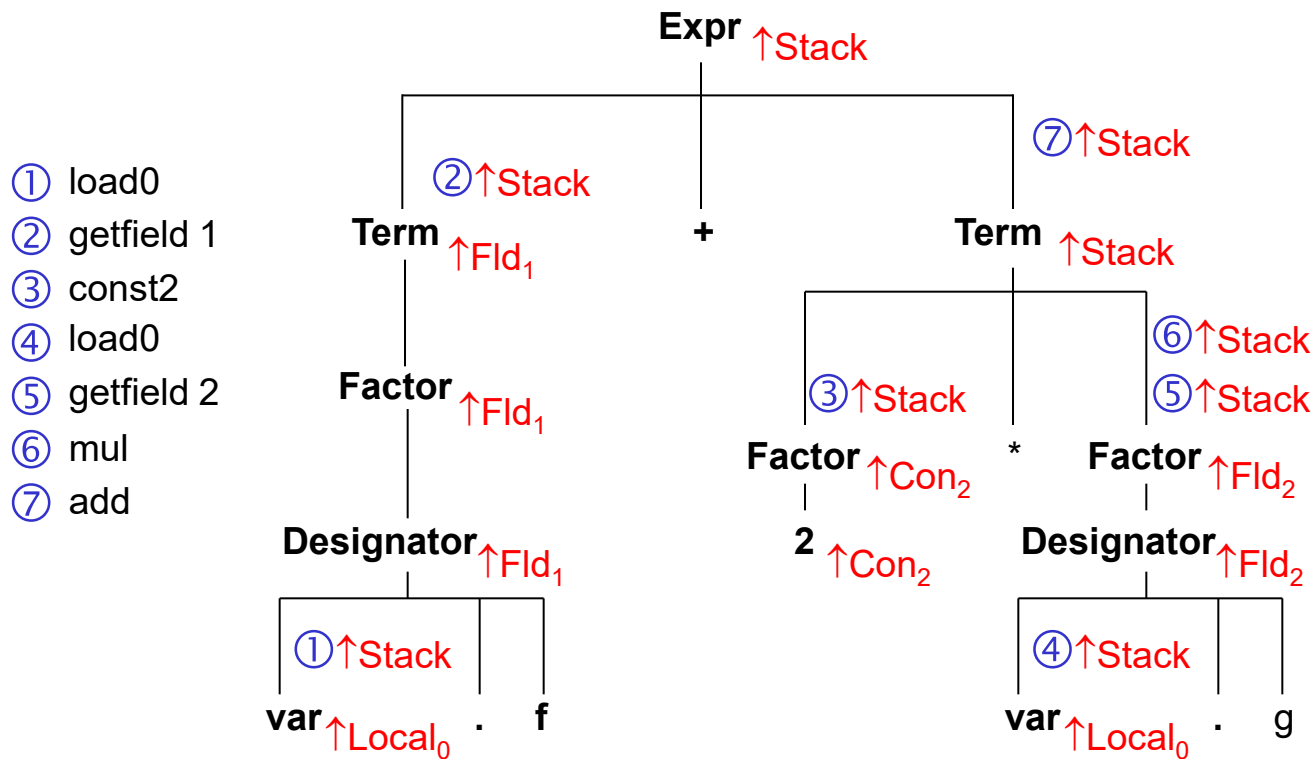
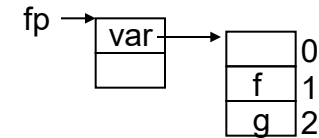
- *ident* must denote a type.
- The type of *Expr* must be *int*.

```
Factor <↑x>      (. Operand x; int val; String name; .)
= Designator <↑x> // for function calls see later
| number <↑val>    (. x = new Operand(val); .)
| charCon <↑val>   (. x = new Operand(val); x.type = Tab.charType; .)
| "(" Expr <↑x> ")"
| "new" ident <↑name> (. Obj obj = Tab.find(name); Struct type = obj.type; .)
( "["              (. if (obj.kind != Obj.Type) error("type expected"); .)
  Expr <↑x> "]"     (. if (x.type != Tab.intType) error("array size must be of type int");
                    Code.load(x);
                    Code.put(Code.newarray);
                    if (type == Tab.charType) Code.put(0); else Code.put(1);
                    type = new Struct(Struct.Arr, type); .)
|                  (. if (obj.kind != Obj.Type || type.kind != Struct.Class)
                    error("class type expected");
                    Code.put(Code.new_); Code.put2(type.nFields);
)                  (. x = new Operand(Operand.Stack, 0, type); .)
.
```

Example



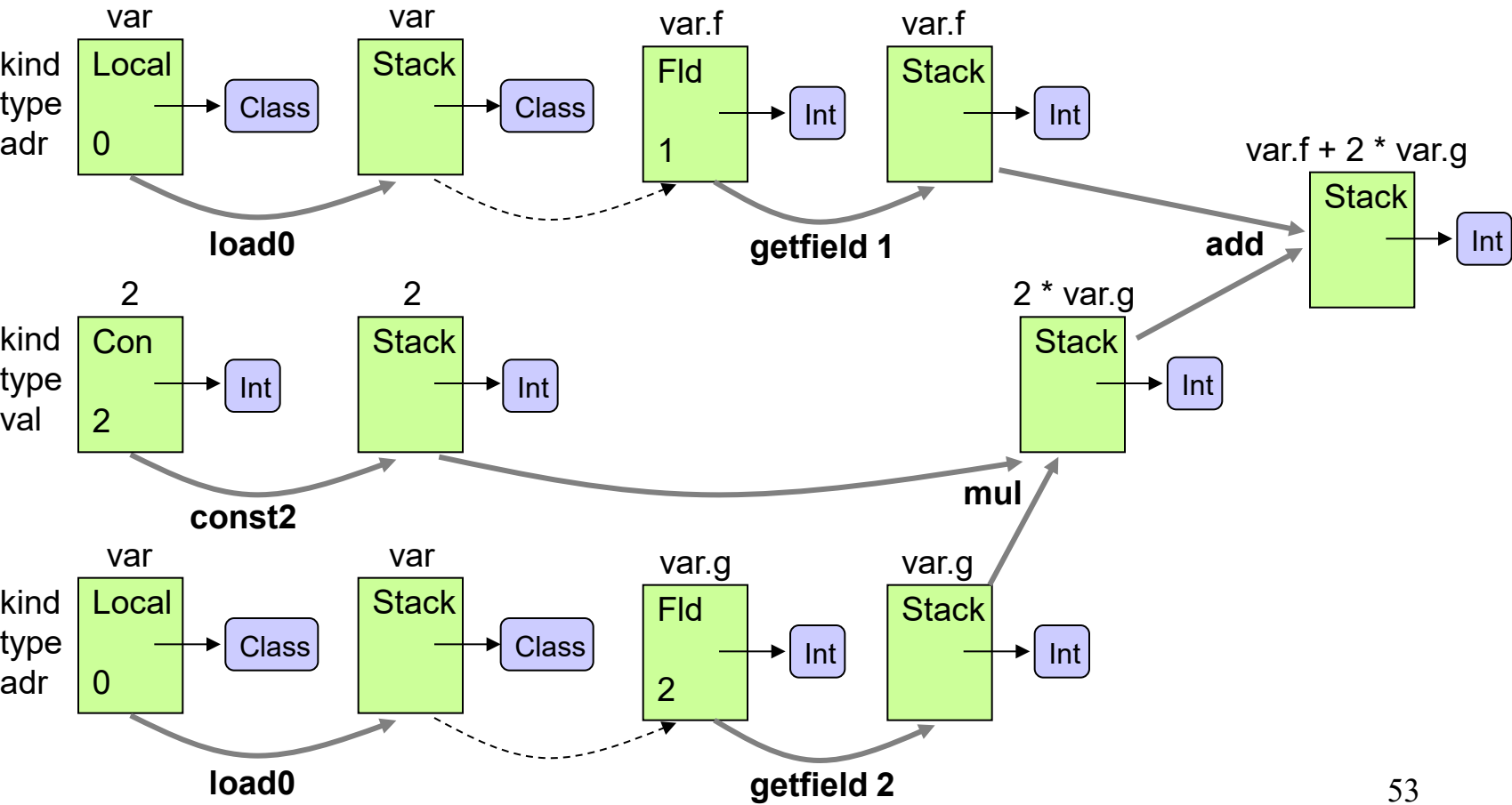
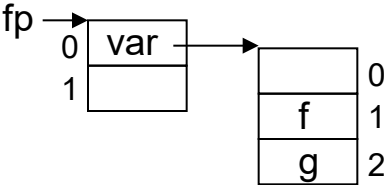
var.f + 2 * var.g





Sequence of operands

var.f + 2 * var.g



6. Code Generation

6.1 Overview

6.2 The MicroJava VM

6.3 Code Buffer

6.4 Operands

6.5 Expressions

6.6 Assignments

6.7 Jumps and Labels

6.8 Control Structures

6.9 Methods

Code patterns for assignments



designator = expr ;

4 cases depending on the kind of the designator on the left-hand side

localVar = expr;	globalVar = expr;	obj.f = expr;	a[i] = expr;
... load expr ... store adr _{localVar}	... load expr ... putstatic adr _{globalVar}	load obj ... load expr ... putfield adr _f	load a load i ... load expr ... astore

The blue instructions are already generated
by *Designator*!

Compiling assignments



Context condition

Statement = Designator "=" Expr ";".

- *Designator* must denote a variable, an array element or an object field.
- The type of *Expr* must be assignment compatible with the type of *Designator*.

Description by an ATG

```
Assignment      (. Operand x, y; .)
= Designator <↑x> // Designator may already generate code
  "=" Expr <↑y>   (. if (y.type.assignableTo(x.type))
                    Code.load(y);
                    Code.assignTo(x); // x: Local | Static | Fld | Elem
                    else
                    error("incompatible types in assignment");
                    .)
  ",".
```

Assignment compatibility

y is assignment compatible with *x*

- if *x* and *y* have the same type (*x.type* == *y.type*), or
- if *x* and *y* are arrays with the same element type, or
- if *x* has a reference type (class or array) and *y* is *null*

6. Code Generation

6.1 Overview

6.2 The MicroJava VM

6.3 Code Buffer

6.4 Operands

6.5 Expressions

6.6 Assignments

6.7 Jumps and Labels

6.8 Control Structures

6.9 Methods

Conditional and unconditional Jumps



Unconditional jumps

```
jmp offset
```

Conditional jumps

```
... load operand1 ...  
... load operand2 ...  
jeq offset
```

if (operand1 == operand2) jmp offset

jcc	jeq	jump on equal
	jne	jump on not equal
	jlt	jump on less than
	jle	jump on less or equal
	jgt	jump on greater than
	jge	jump on greater or equal

static final int

```
eq = 0,  
ne = 1,  
lt = 2,  
le = 3,  
gt = 4,  
ge = 5;
```

in class *Code*

Generation of jump instructions

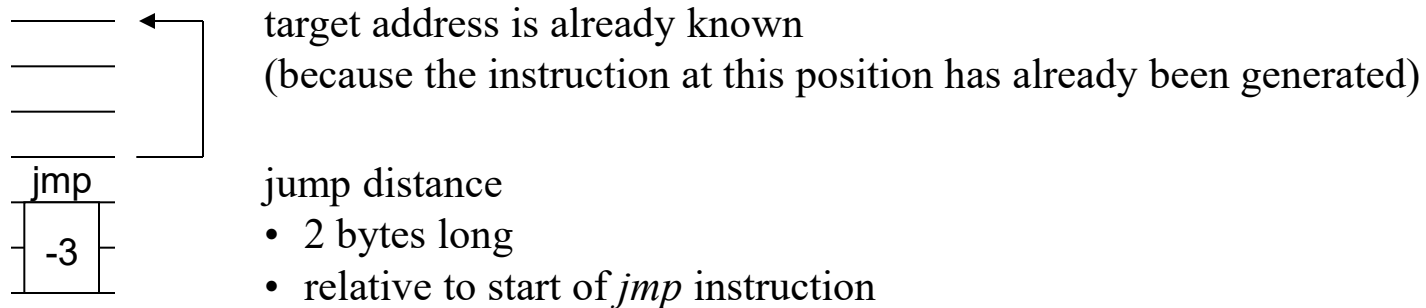
```
Code.put(Code.jmp);  
Code.put2(offset);
```

```
Code.put(Code.jcc + operator);  
Code.put2(offset);
```

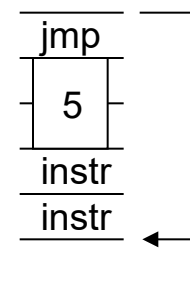
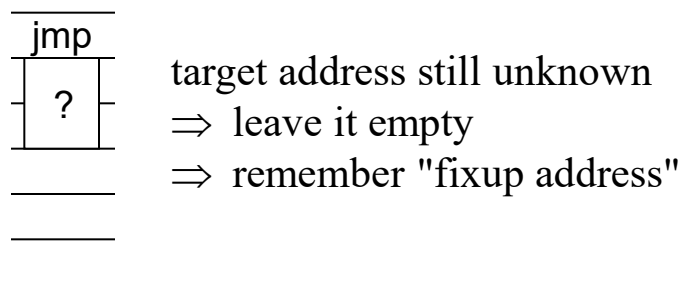


Forward and backward Jumps

Backward jumps



Forward jumps



patch it when the target address becomes known (fixup)

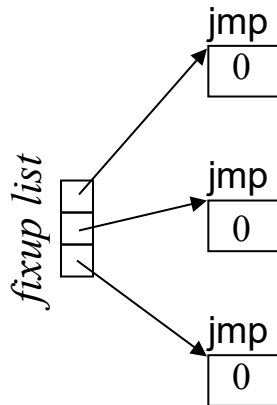
Forward jumps to the same jump target



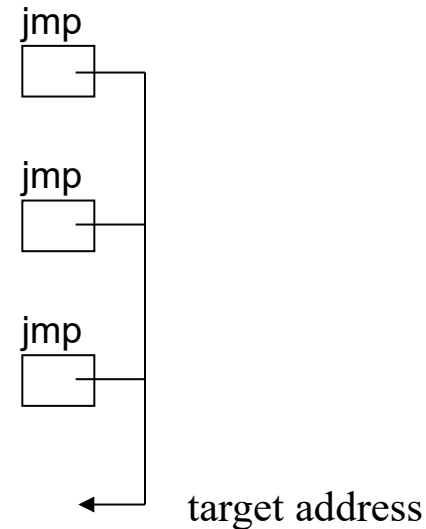
Where does this occur?

- breaks in a loop or in a switch statement
- Boolean expressions that are built with `&&` and `||` (see later)

Open jumps to the same target
are kept in a *fixup list*



When the target becomes known
the fixup list is resolved





Class Label

Manages jump labels and fixup lists

```
class Label {  
    Label() {...}           // creates a new and still undefined label  
    void here() {...}       // defines a label at the current pc position  
    void putAdr() {...}     // emits jump distance to this label at the pc position  
}
```

Usage

```
Label label = new Label();  
...  
Code.put(Code.jump);      // jump to a yet undefined label  
label.putAdr();           // fixup address is remembered in label  
..  
label.here();              // the jump to label leads to here
```

Implementation of the class Label



Internal state of a label

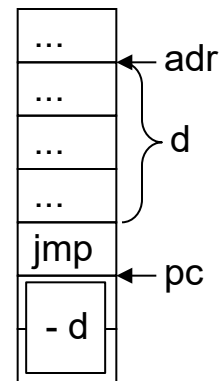
```
class Label {  
    private int adr; // adr >= 0: label address (already defined)  
                    // adr < 0: label still undefined  
    private ArrayList<Integer> fixupList; // fixup addresses  
    ...  
}
```

Constructor

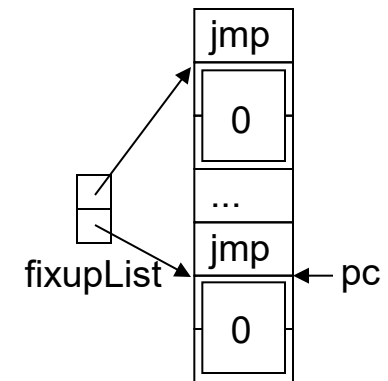
```
public Label() { adr = -1; fixupList = new ArrayList<Integer>(); }
```

putAdr()

```
public void putAdr() {  
    if (adr >= 0)  
        Code.put2(adr - (Code.pc-1));  
    else {  
        fixupList.add(Code.pc);  
        Code.put2(0);  
    }  
}
```



if (adr >= 0) ...



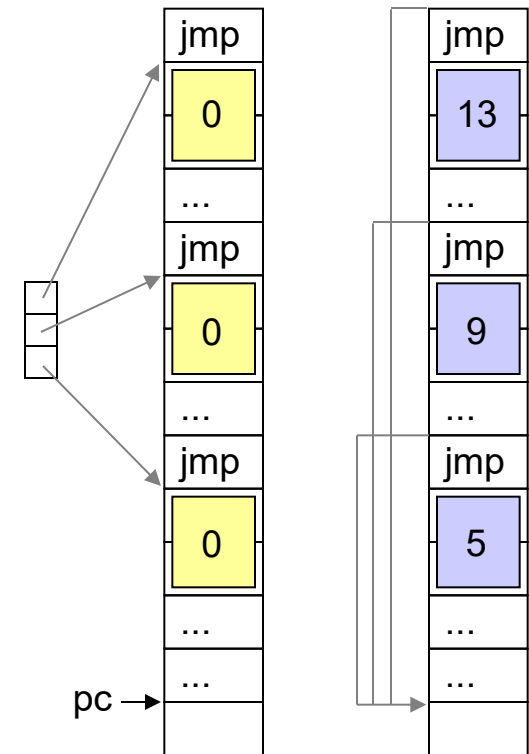
if (adr < 0) ...

Implementation of the class Label



here()

```
public void here() {  
    if (adr >= 0) error("label defined twice");  
    for (int pos: fixupList) {  
        Code.put2(pos, Code.pc - (pos-1));  
    }  
    adr = Code.pc;  
}
```



New method in class *Code*

`Code.put2(pos, val);` writes *val* (2 bytes) to the address *pos* in the code buffer

Conditions



if (a > b) ...

Condition

code pattern

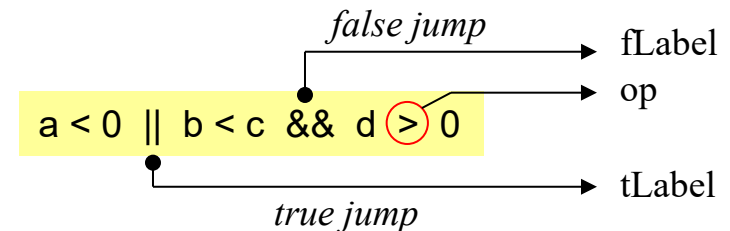
load a
load b
jle ...

- Problem: there is no compare instruction in the μ JVM (the comparison only happens in the jump instruction)
- *Condition* thus cannot do the comparison
=> returns a compare operator instead

Cond operand

Condition returns a new operand kind with the following fields:

- compare operator: eq, ne, lt, le, gt, ge
 - labels (fixup lists) for jumps out of the *Condition*
 - tLabel: for *true jumps*
 - fLabel: for *false jumps*
- necessary for conditions that are built with && and ||



True jumps and false jumps

true jump jumps if the condition is true $a > b \Rightarrow \text{jgt ...}$
false jump jumps if the condition is false $a > b \Rightarrow \text{jle ...}$

Cond operands



```
class Operand {  
    static final int Con=0, Local=1, Static=2, Stack=3, Fld=4, Elem=5, Meth=6, Cond=7;  
  
    int    kind;    // Con, Local, Static, ...  
    Struct type;    // operand type  
    int    val;     // Con: constant value  
    int    adr;     // Local, Static, Fld, Meth: address  
    Obj    obj;     // Meth: method object  
    int    op;      // Cond: compare operator  
    Label  tLabel;   // target label (fixup list) for true jumps  
    Label  fLabel;   // target label (fixup list) for false jumps  
}
```

Additional constructor (for Cond operands)

```
public Operand (int kind, int val, Struct type) {  
    this.kind = kind; this.val = val; this.type = type;  
    if (kind == Cond) {  
        op = val;  
        tLabel = new Label();  
        fLabel = new Label();  
    }  
}
```



Generating unconditional jumps

Using an auxiliary method in class *Code*

```
class Code {  
    ...  
    public static void jump(Label lab) {  
        put(jmp); lab.putAdr();  
    }  
    ...  
}
```

Usage

```
Label label = new Label();  
...  
Code.jump(label);
```



Generating conditional jumps

Using auxiliary methods in class *Code*

```
class Code {  
    private static final int eq = 0, ne = 1, lt = 2, le = 3, gt = 4, ge = 5;  
    private static int inverse[] = {ne, eq, ge, gt, le, lt};  
    ...  
    public static void tJump (int op, Label label) {  
        put(jcc + op); // jeq, jne, jlt, jle, jgt, jge  
        label.putAdr();  
    }  
  
    public static void fJump (int op, Label label) {  
        put(jcc + inverse[op]); // jne, jeq, jge, jgt, jle, jlt  
        label.putAdr();  
    }  
    ...  
}
```

Usage

```
"if" "(" Condition <↑x> ")"      (. Code.fJump(x.op, x.fLabel); .)
```

6. Code Generation

6.1 Overview

6.2 The MicroJava VM

6.3 Code Buffer

6.4 Operands

6.5 Expressions

6.6 Assignments

6.7 Jumps and Labels

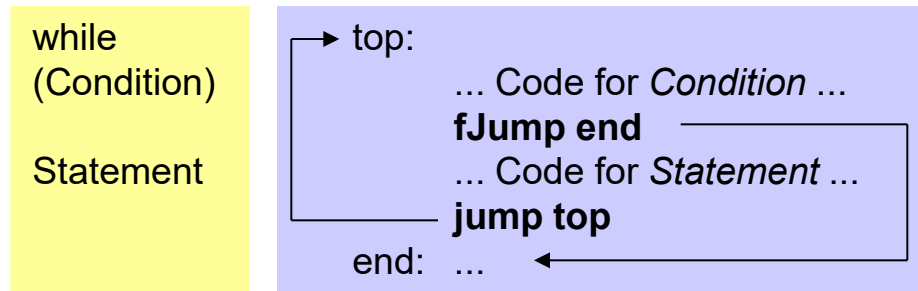
6.8 Control Structures

6.9 Methods



While statement

Desired code pattern



Description by an ATG

WhileStatement	(. Operand x; .)
= "while"	(. Label top = new Label();
	top.here(); .)
"(" Condition <↑x> ")"	(. Code.fJump(x.op, x.fLabel); .)
Statement	(. Code.jump(top);
	x.fLabel.here(); .)
.	

Example

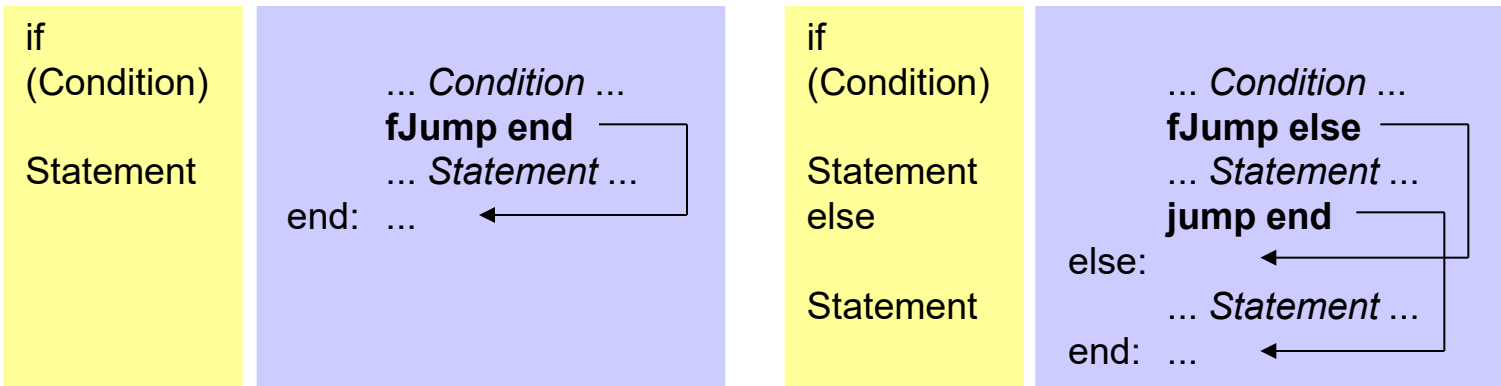
```
while (a > b) a = a - 2;

10 load0 ← top
11 load1
12 jle 10 (=22)
15 load0
16 const2
17 sub
18 store0
19 jmp -9 (=10)
22 ... ← x.fLabel
```



If statement

Desired code pattern



Description by an ATG

```
IfStatement          (. Operand x; Label end; .)
= "if"
  "(" Condition <↑x> ")"  (. Code.fJump(x.op, x.fLabel); .)
  Statement
  ( "else"                (. end = new Label();
                           Code.jump(end);
                           x.fLabel.here(); .)
    Statement             (. end.here(); .)
    |                     (. x.fLabel.here(); .)
  ).
```

Example

if (a > b) max = a; else max = b;

```
10 load0
11 load1
12 jle 8(=20)
15 load0
16 store2
17 jmp 5(=22)
20 load1
21 store2
22 ...
```

x.fLabel
end



Works also with nested if statements

```
IfStatement      (. Operand x; Label end; .)
= "if"
  "(" Condition <↑x> ")" (. Code.fJump(x.op, x.fLabel); .)
  Statement
  ( "else"          (. end = new Label();
                    Code.jump(end);
                    x.fLabel.here(); .)
    Statement      (. end.here(); .)
  |
  ).
```

if (c1)

if (c2)

s1

else

s2

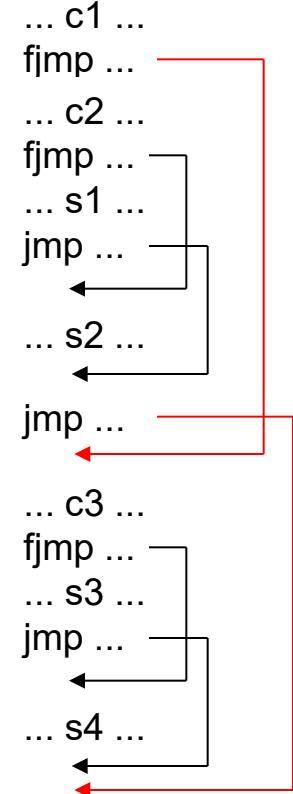
else

if (c3)

s3

else

s4





Break statement

Exit from loops

- Set a label *breakLab* after the end of the loop
- When a *break* statement occurs: `Code.jump(breakLab);`

Nested loops

- Each loop needs its own *breakLab*
- *breakLab* must be saved using a stack

```
Statement
= "while"
    "(" Condition <↑x> ")"
    Statement
| "break" ";"
| ... .

( // global declarations
  Label breakLab = null;
  Stack<Label> breaks = new Stack<>(); .)

( breaks.push(breakLab);
  breakLab = new Label(); ... .)
( ... .)
( ...
  breakLab.here();
  breakLab = breaks.pop(); .)

( if (breakLab == null) error("break outside a loop");
  Code.jump(breakLab); .)
```

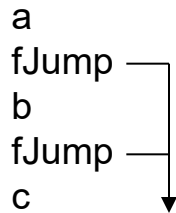
Question: What would be necessary to use *break* with a named label?

Short-circuit evaluation of Boolean expressions

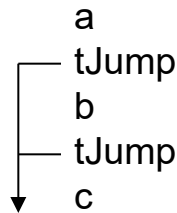


- Boolean expressions can be built with && and ||
- Evaluation of the expression stops as soon as the result is known

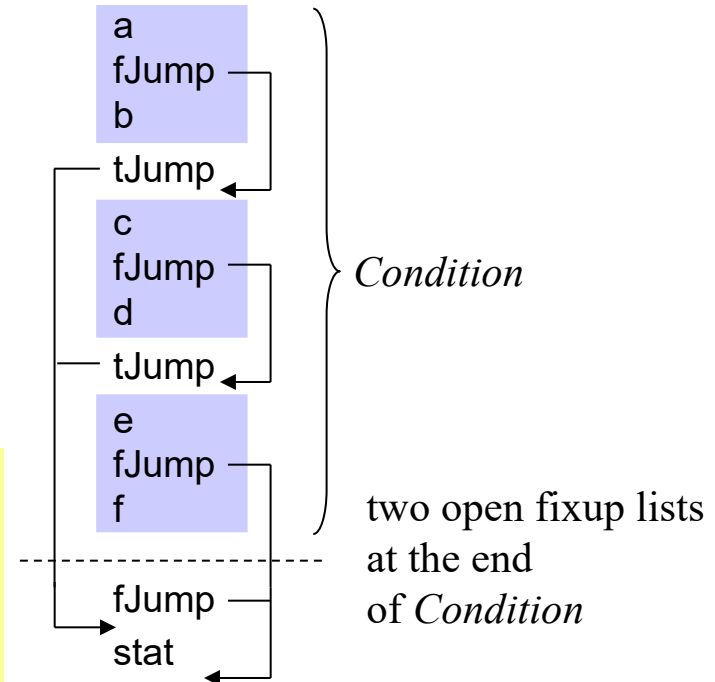
a && b && c



a || b || c



if (a && b || c && d || e && f) stat;



Small modification in the ATG of if statements

```

IfStatement
= "if"
  "(" Condition <↑x> ")"      (. Code.fJump(x.op, x.fLabel);
                             x.tLabel.here(); .)
  Statement                  (. x.fLabel.here(); .)
  .
    
```

similar modification is necessary in the while statement



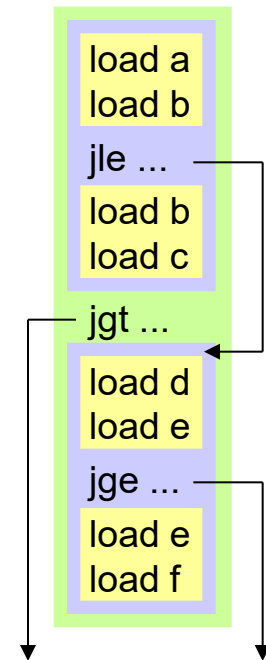
Compiling Boolean expressions

```
CondFactor <↑x>    (. Operand x, y; int op; .)
= Expr <↑x>          (. Code.load(x); .)
  Relop <↑op>
  Expr <↑y>           (. Code.load(y);
    if (!x.type.compatibleWith(y.type))
      error("type mismatch");
    if (x.type.isRefType()
        && op != Code.eq && op != Code.ne)
      error("invalid compare");
    x = new Operand(Operand.Cond, op, null);
    .) .
```

```
CondTerm <↑x>      (. Operand x, y; .)
= CondFactor <↑x>
  { "&&"             (. Code.fJump(x.op, x.fLabel); .)
    CondFactor <↑y>  (. x.op = y.op; .)
  }.
```

```
Condition <↑x>     (. Operand x, y; .)
= CondTerm <↑x>
  { "||"             (. Code.tJump(x.op, x.tLabel);
    x.fLabel.here(); .)
    CondTerm <↑y>    (. x.op = y.op; x.fLabel = y.fLabel; .)
  }.
```

a>b && b>c || d<e && e<f



6. Code Generation

6.1 Overview

6.2 The MicroJava VM

6.3 Code Buffer

6.4 Operands

6.5 Expressions

6.6 Assignments

6.7 Jumps and Labels

6.8 Control Structures

6.9 Methods



Method call

Code pattern

<code>m(a, b);</code>	load a load b call m	parameters are passed on the <i>EStack</i>
-----------------------	----------------------------	--

Description by an ATG

Statement	(. Operand x, y;)
= Designator <↑x>	
(ActPars <↓x>	(. Code.callMethod(x);
"=" Expr <↑y>	if (x.type != Tab.noType) Code.put(Code.pop); .)
)	(.)
"."	
,	
... .	



Function call

Code pattern

```
c = m(a, b);
```

load a parameters are passed on the *EStack*
load b
call m
store c function value is returned on the *EStack*

Description by an ATG

```
Factor <↑x>      (. Operand x; .)
= Designator <↑x>
  [ ActPars <↓x>      (. if (x.type == Tab.noType) error("procedure called as a function");
                     Code.callMethod(x);
                     x.kind = Operand.Stack; .)

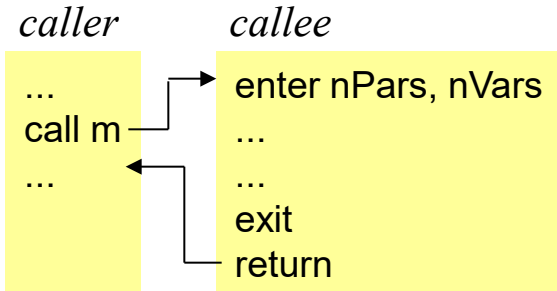
  ]
| ...
```

```
static void callMethod (Operand m) {
  if (m.obj == Tab.ordObj || m.obj == Tab.chrObj) ; // nothing
  else if (m.obj == Tab.lenObj)
    Code.put(Code.arraylength);
  else {
    Code.put(Code.call);
    Code.put2(m.adr - (Code.pc-1));
  }
}
```

Standard functions

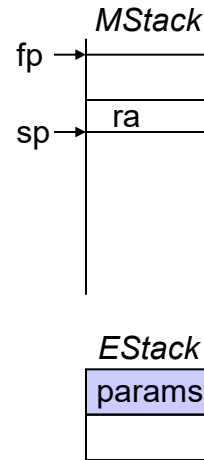
- ord('a')
- *ActPars* loads 'a' onto the *EStack*
 - loaded value gets the type of *ordObj* (= *intType*) and *kind* = *Operand.Stack*

Stack frames



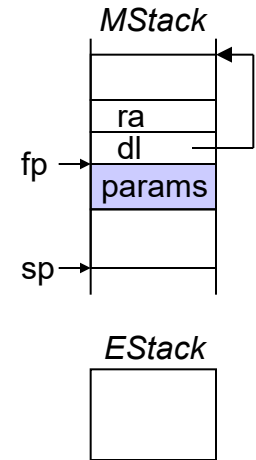
enter ... creates a stack frame
exit ... removes a stack frame

Method entry

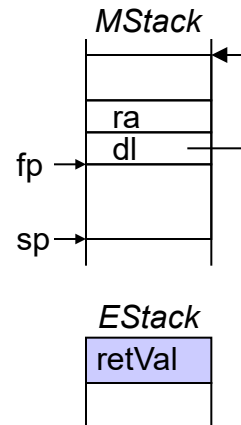


enter nPars, nVars

```
PUSH(fp); // dynamic link
fp = sp;
sp = sp + nVars;
initialize frame to 0;
for (i = nPars-1; i >= 0; i--)
    local[i] = pop();
```

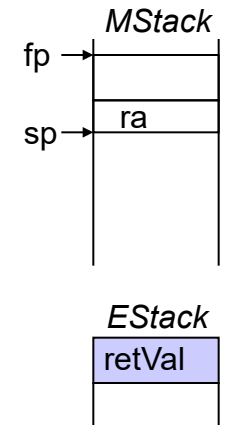


Method exit



exit

```
sp = fp;
fp = POP();
```



Method declaration



MethodDecl	(. Struct type; String name; int n; .)
= (Type <↑type>	(. if (type.isRefType()) error("methods may only return int or char"); .)
"void"	(. type = Tab.noType; .)
)	
ident <↑name>	(. curMethod = Tab.insert(Obj.Meth, name, type);
	Tab.openScope(); .)
("(" FormPars <↑n> ")")	(. curMethod.nPars = n;
	if (name.equals("main")) {
	Code.mainPc = Code.pc;
	if (curMethod.type != Tab.noType) error("method main must be void");
	if (curMethod.nPars != 0) error("main must not have parameters");
	} .)
{ VarDecl }	(. curMethod.locals = Tab.curScope.locals;
	curMethod.adr = Code.pc;
	Code.put(Code.enter);
	Code.put(curMethod.nPars);
	Code.put(Tab.curScope.nVars); .)
Block	(. if (curMethod.type == Tab.noType) {
	Code.put(Code.exit);
	Code.put(Code.return_);
	} else { // end of function reached without a return statement
	Code.put(Code.trap); Code.put(1);
	}
	Tab.closeScope(); .)
.	



Formal parameters

- are entered into the symbol table (as variables of the method scope)
- their number is counted

```
FormPars <↑n>      (. int n = 0; .)
= [  FormPar        (. n++; .)
    { ", " FormPar   (. n++; .)
      }
  ].
```

```
FormPar            (. Struct type; String name; .)
= Type <↑type>
  ident <↑name>      (. Tab.insert(Obj.Var, name, type); .)
.
```



Actual parameters

- load them onto the *EStack*
- check if they are assignment compatible with the formal parameters
- check if the numbers of actual and formal parameters match

```
ActPars <↓m>      (. Operand m, ap; .)
= "("              (. if (m.kind != Operand.Meth) { error("not a method"); m.obj = Tab.noObj; }
                    int aPars = 0;
                    int fPars = m.obj.nPars;
                    Obj fp = m.obj.locals; .)

  [ Expr <↑ap>      (. Code.load(ap); aPars++;
                    if (fp != null) {
                      if (!ap.type.assignableTo(fp.type)) error("parameter type mismatch");
                    } .)

    { ", " Expr <↑ap> (. Code.load(ap); aPars++;
                      fp = fp.next;
                      if (fp != null) {
                        if (!ap.type.assignableTo(fp.type)) error("parameter type mismatch");
                      } .)

  }

]                  (. if (aPars > fPars)
                    error("too many actual parameters");
                    else if (aPars < fPars)
                      error("too few actual parameters"); .)

)" " .
```

Return statement



Statement

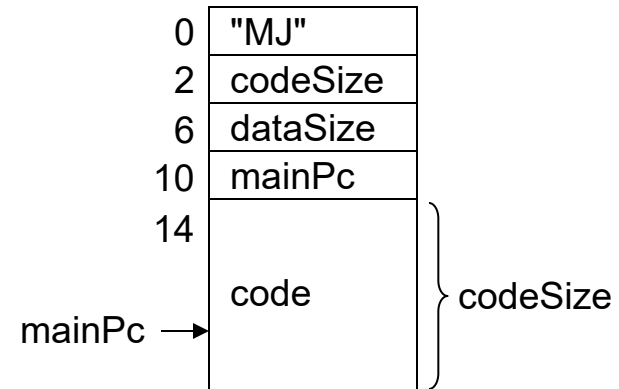
```
= ...  
| "return"  
  ( Expr <↑x>      (. Code.load(x);  
                    if (curMethod.type == Tab.noType)  
                      error("void method must not return a value");  
                    else if (!x.type.assignableTo(curMethod.type))  
                      error("type of return value must match method type");  
                    .)  
  |                (. if (curMethod.type != Tab.noType) error("return value expected"); .)  
  )                (. Code.put(Code.exit);  
                    Code.put(Code.return_); .)  
  ", "  
  , .
```

Object file



Contents of the object file in MicroJava

- information for the loader
 - code size (in bytes)
 - size of the global data area (in words)
 - address of the *main()* method
- code



Further contents for other languages (Java, C, Pascal, ...)

- String constant area
- List of exported symbols ({name address})
- Occurrence of imported symbols in the code (fixup information)
 { name {codeAddress} }
- Metainformation (for debugging and dynamic loading)
- ...